From Tabucol to Ant Colony Optimization: Transferring Reinforcement-Learning Guidance for Graph Coloring

Gaoyuan Zhang[1], Enze Cai[1], and Omar Dib[1⋆]

Wenzhou-Kean University, Wenzhou, China
1306316@wku.edu.cn, Caie@kean.edu, odib@kean.edu

Abstract. We investigate whether reinforcement-learning guidance trained in a Tabucol pipeline can be transferred to a population-based search and then used to improve the final local search. The study proceeds in four stages. We compare RLTabucol with pure Tabucol and a matched random-policy baseline, diagnose a representative failure with an actor-only experiment, and transfer the frozen policy to ACO through soft pheromone injection. We then pass the converged RLACO solution to Tabucol under the same five-hour refinement budget used by the source baselines. RLTabucol is equal to or better than RLTabucol_0 on eight of eleven instances, while the transfer experiments show that RLACO improves the random-policy baseline on the reported instances and that soft injection is competitive with conflict-based constructed injection. The terminal table shows that RLACO+Tabucol reaches lower color budgets on several difficult instances, including DSJC500.9, `le450_15c`, `le450_15d`, and `le450_25d`. These results support a staged division of labor: the policy supplies structured guidance, ACO supplies population diversity, and Tabucol performs final local repair.

Keywords: Graph coloring · Reinforcement learning · Ant colony optimization · Policy transfer · Tabucol

1 Introduction

Graph coloring requires both effective conflict reduction and enough diversification to escape difficult search basins. Tabucol is a strong single-solution local search for fixed-k coloring, whereas ant colony optimization (ACO) constructs a population of colorings and maintains pheromone-based diversity. Reinforcement learning (RL) can learn vertex–color preferences, but a policy that is useful inside one search framework may not provide the right starting states for another.

This paper studies that interface directly. We ask whether a policy trained with Tabucol can be transferred to ACO, and whether the resulting ACO solution can then be refined by Tabucol. The question is motivated by an empirical pattern: the policy can reduce conflicts, yet its direct handoff to Tabucol is not

⋆ Corresponding author

uniformly beneficial. A population-based intermediate stage may provide the diversity missing from the direct handoff while retaining the policy's problem-specific preferences.

We organize the evaluation as a sequence of increasingly targeted tests. First, RLTabucol is compared with pure Tabucol and the matched random-policy baseline RLTabucol_0. Second, an actor-only diagnostic examines a persistent failure case. Third, RLACO is compared with RLACO_0, pure ACO, and the conflict-based constructed-injection variant $\text{RLACO}_{\text{config}}$. Finally, the converged RLACO solution is refined by Tabucol under the same five-hour Tabucol budget used for the source baselines.

The contributions are:

1. We establish a controlled source-framework comparison showing that the learned policy is useful on most tested instances, but its benefit is instance-dependent.
2. We use an actor-only diagnostic to explain the failure regime: the actor performs coarse conflict reduction, whereas Tabucol provides the dominant fine-grained repair and can stagnate when the handoff lacks suitable diversity.
3. We evaluate cross-framework policy transfer through soft pheromone injection, with pure ACO, RLACO_0, and $\text{RLACO}_{\text{config}}$ as complementary controls.
4. We propose RLACO+Tabucol as the main optimization framework. It uses the transferred policy and ACO to construct diverse, policy-informed starting states, then gives Tabucol the five-hour budget needed for final repair.

The remainder of the paper reviews related work, defines the policy and search configurations, describes the experimental protocol, reports the staged results, and discusses limitations.

2 Related Work

2.1 Local search for graph coloring

Tabu search for graph coloring was introduced by Hertz and de Werra [6] and remains a strong single-solution method for fixed-color feasibility. Tabucol repeatedly selects admissible vertex–color moves and uses tabu memory to avoid immediate reversals. Hybrid methods have embedded Tabucol in genetic and adaptive-memory frameworks [4,5], while surveys have documented the broader range of coloring heuristics [9]. These methods are effective at fine local repair, but a single incumbent can have difficulty escaping a narrow near-feasible basin. Our terminal stage uses this strength after a population-based construction stage has supplied a more diverse starting state.

2.2 Ant colony optimization for coloring

ACO approaches construct colorings incrementally from pheromone and heuristic information. Costa and Hertz [2] adapted the construction paradigm to graph coloring, and later work has combined ACO with prediction or learned heuristics [12,16]. These approaches generally learn or tune guidance inside the target ACO framework. Our setting is different: the policy is trained with Tabucol and transferred without retraining. The comparison with pure ACO isolates the value of adding the transferred prior, while the $\text{RLACO}_{\text{config}}$ comparison isolates the choice of a soft distributional interface.

2.3 Learning-guided combinatorial search

RL has been used to guide coloring through self-play, value-based methods, graph neural networks, and learned ordering heuristics [7,14,8,13,15]. Hybrid RL–local-search methods similarly use a policy to propose moves that a classical heuristic refines [1,3]. Most evaluations remain within the framework used for training. The present study treats cross-framework transfer as the object of evaluation: it asks whether vertex–color preferences learned for local search remain useful when converted into a pheromone prior for construction.

The resulting contribution is a staged interaction between the two paradigms. The RL phase supplies learned preferences and coarse conflict reduction, ACO supplies population diversity, and Tabucol supplies fine repair. The source-framework, transfer, injection, phase-share, and terminal experiments are designed to distinguish these roles rather than collapse them into one aggregate score.

3 Methodology

This section presents the shared policy and the evaluation configurations used in the study. We first describe the learned recoloring policy that serves as the shared guidance module, including its actor and critic network architectures. We then introduce the RLTabucol pipeline, which we implement by coupling this policy with Tabucol local search. Next, motivated by the instance-dependent results of the RLTabucol experiment, we define an actor-only diagnostic that isolates the policy's behavior by removing the Tabucol handoff. Finally, we present the RLACO pipeline, which transfers the trained policy to ACO through a soft pheromone-injection interface that we design for studying cross-framework transfer. Figure 1 provides a visual summary of the policy, source-framework, transfer, and terminal configurations.

3.1 Learned recoloring policy

The core of all evaluation configurations is a learned policy that selects vertex–color recoloring actions. The coloring task is modeled as a finite-horizon Markov

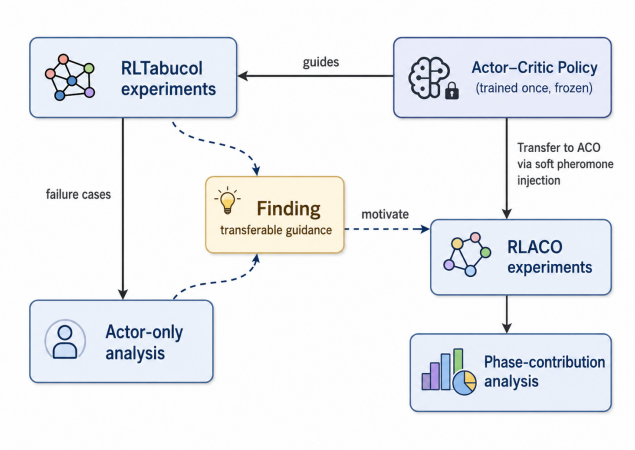


Fig. 1. Overview of the three evaluation configurations and their relationships. The learned actor–critic policy is trained once in RLTabucol and then reused across the source, transfer, and terminal paths.

decision process with a fixed horizon of I steps. At each step t, the state consists of the graph $G = (V, E)$, the color budget k, and the current coloring c_t. An action is a pair $a_t = (v_t, q_t)$ specifying a vertex and a target color. The reward is the reduction in the number of conflicting edges, $r_t = f(c_t) - f(c_{t+1})$, where $f(c)$ counts edges whose endpoints share the same color.

The policy is implemented as an actor–critic architecture. The actor network factorizes the joint action into two components: a vertex-selection distribution and a conditional color-selection distribution:

$$\pi_{\theta,\phi}((v, q) \mid s) = \pi_\theta^{\mathrm{v}}(v \mid s)\, \pi_\phi^{\mathrm{c}}(q \mid s, v). \tag{1}$$

The vertex-selection head π_θ^{v} is a two-layer multilayer perceptron ($3 \to 16 \to 1$ with ReLU) applied independently to every candidate vertex, followed by a softmax over all vertices. It uses three vertex-level features: degree, conflicting edge count, and number of distinct neighbor colors. The color-selection head π_ϕ^{c} is a separate two-layer MLP ($3 \to 16 \to 1$ with ReLU) applied to every vertex–color pair, followed by a softmax over colors. It uses three color-level features: number of conflicting neighbors, class size, and total vertex count. Because each MLP processes only local features without graph-level pooling, the actor network is permutation-equivariant by construction through local vertex features, and generalizes across graph sizes and color budgets.

The critic network shares the same feature extraction structure as the actor and appends a linear layer over the flattened $n \times k$ logit matrix to produce a scalar state value $V_\psi(s)$. This value is used to compute advantage estimates for Proximal Policy Optimization (PPO) [10], which updates both the actor and critic parameters using the standard clipped surrogate objective. During

RLTabucol training, the reward combines immediate per-step conflict-reduction rewards with the final reward returned by Tabucol. The default policy is trained on $G(n{=}250, p{=}0.5)$ random graphs with $k{=}24$ colors. Once trained, the policy parameters are frozen and reused across all three evaluation configurations without further updates. Detailed PPO and training hyperparameters are reported in Appendix A.

3.2 The RLTabucol pipeline

We implement the RLTabucol pipeline by interleaving the learned policy with Tabucol local search within an episodic loop [1,3]. Each run begins with a random k-coloring. Within each episode, the actor performs I sequential recoloring steps, producing an intermediate coloring that is then passed to Tabucol. Tabucol is a single-solution local search that iteratively selects the best admissible vertex–color recoloring move, subject to a tabu list that prevents recently reversed moves. It runs for at most T iterations and returns both a refined coloring and the cumulative reward accumulated by its internal moves. The detailed Tabucol procedure is given in Algorithm 3 in Appendix A.

The coloring carried into the next episode is selected probabilistically: with probability β the current coloring is kept, and with probability $1{-}\beta$ the best coloring found during that Tabucol call is returned instead. Algorithm 1 summarizes the complete RLTabucol pipeline. During training, this loop runs for 50 epochs and the collected trajectories are used for PPO updates. During testing, the actor parameters are fixed and the loop continues until a zero-conflict coloring is found or the five-hour wall-clock budget is exhausted.

To verify the effectiveness of the RLTabucol framework and the trained policy, we define a matched baseline called RLTabucol_0 that replaces the trained actor with a uniform random policy over the same action space. All other components—episode structure, reward computation, Tabucol procedure, return rule, and time budget—remain identical.

3.3 Actor-only diagnostic

The instance-dependent results of the RLTabucol experiment (Section 5.1) motivate a closer examination of the policy behavior on failure cases. We therefore define an actor-only configuration that removes Tabucol entirely. In this setting, each episode consists solely of I RL recoloring steps, and the running-best score is tracked across episodes without any local search refinement. The diagnostic is applied to `le450_15d` at $k{=}15$, a persistent failure case for the default policy. By comparing the actor-only trajectory against the full RLTabucol pipeline under the same time budget, we can contrast the observed trajectories and diagnose the operating regime in which the Tabucol handoff is associated with additional conflict reduction.

Algorithm 1 The RLTabucol pipeline

Input: G, k, actor $\pi_{\theta,\phi}$, steps I, Tabucol iters T, prob. β, budget $\mathcal{B}$
$c \leftarrow$ random k-coloring; $c^\star \leftarrow c$
while $\mathcal{B}$ not exhausted **do**
  for $t = 0, \ldots, I-1$ **do**
    $(v, q) \sim \pi_{\theta,\phi}(\cdot \mid (G, k, c))$; $c \leftarrow c[v \mapsto q]$
    if $f(c) < f(c^\star)$ **then**
      $c^\star \leftarrow c$
    end if
    if $f(c^\star) = 0$ **then**
      return $c^\star$
    end if
  end for
  $c \leftarrow \textsc{Tabucol}(G, k, c, T, \beta)$
  if $f(c) < f(c^\star)$ **then**
    $c^\star \leftarrow c$
  end if
  if $f(c^\star) = 0$ **then**
    return $c^\star$
  end if
end while
return $c^\star$

3.4 Evaluation configurations and terminal refinement

In addition to RLTabucol and its random-policy control RLTabucol_0, we evaluate pure Tabucol as the local-search reference. For the transfer study, pure ACO is run with the same ACO settings, RLACO_0 replaces the trained policy with a random policy, and $\text{RLACO}_{\text{config}}$ uses the conflict-based constructed injection. RLACO uses the soft distributional injection defined below. For the terminal experiment, the converged RLACO coloring is passed to Tabucol without retraining. The RLACO stage is allowed its instance-dependent convergence time; the terminal Tabucol stage uses a five-hour wall-clock budget, matching the three Tabucol-based baselines.

3.5 Terminal RLACO+Tabucol handoff

The terminal configuration reuses the frozen policy and does not perform additional training. RLACO runs until the instance-dependent transfer budget is reached and returns its incumbent coloring. Tabucol is initialized with this incumbent and then runs under a five-hour wall-clock limit, the same limit used by pure Tabucol, RLTabucol, and RLTabucol_0. The terminal score is the final running-best conflict count, and a run is successful only when this score is zero. This design tests whether a policy-informed, population-explored state gives Tabucol a better starting point for its dominant local repair.

3.6 Transfer to ACO via soft pheromone injection

Having observed that the policy can provide useful guidance in RLTabucol on several instances, we ask whether the same policy can also benefit a different major class of search algorithms, namely population-based construction methods such as ACO. The RLACO pipeline tests this by using the policy's color-selection distribution to bias the pheromone matrix of an ant colony optimization procedure. This transfer is the main contribution of this work: it tests whether learned guidance from a local-search-based framework can benefit a population-based, construction-based framework.

The ACO component follows a MAX–MIN ant system [11] with pheromone clipping, elite reinforcement, a pseudo-random-proportional construction rule, and no ACO-internal local search. An ant constructs a coloring by visiting vertices one by one and selecting colors according to the standard construction rule that combines pheromone and heuristic information. The detailed ACO procedure is given in Algorithm 4 in Appendix A.

The key interface that enables the RLACO study is the soft pheromone injection, which we design to bridge the actor's output and the ACO construction process. Given the current state s, the color-selection head of the actor produces $P(v, q) = \pi^{\mathrm{c}}_{\phi}(q \mid s, v)$, representing the actor's preference for assigning color q to vertex v. As a softmax output, $P(v, \cdot)$ is normalized over colors, i.e., $\sum_{q=1}^{k} P(v, q) = 1$ for every vertex v. At the beginning of each ACO call, after the RL phase, the pheromone matrix is reset to the uniform base level τ_0, and the policy distribution is injected through a soft interpolation:

$$\tau(v, q) \leftarrow \tau_0 + \big(\min(\tau_{\max},\, \sigma\tau_0) - \tau_0\big)\, P(v, q), \tag{2}$$

where $\sigma > 1$ controls the injection strength. The resulting pheromone values are immediately clipped to $[\tau_{\min}, \tau_{\max}]$. The actor therefore provides a soft prior rather than a deterministic assignment: every ant still constructs a coloring from scratch using the biased pheromone, while the RL handoff coloring is retained only as the incumbent passed to ACO. After each ACO iteration, standard evaporation and elite reinforcement are applied, followed by clipping to the same bounds. The policy thus affects ACO only through the initial pheromone matrix and the incumbent at the start of each ACO call. Algorithm 2 summarizes the complete RLACO pipeline.

ACO local-search post-processing is deliberately disabled in RLACO. This design choice serves two purposes. First, it isolates the effect of the pheromone prior: with local search enabled, any observed difference between RLACO and RLACO_0 could be attributed to the local search amplifying small initial differences rather than to the policy's guidance itself. Second, it makes RLACO a controlled transferability test rather than a complete solver comparison; the goal is to determine whether the trained policy provides useful guidance, not to achieve state-of-the-art results.

RLACO_0 is a matched baseline that replaces the trained actor with a random policy while preserving the same RL steps, handoff mechanism, ACO procedure, and time budget. Comparing RLACO with RLACO_0 tests whether the trained

Algorithm 2 The RLACO pipeline

Input: G, k, actor $\pi_{\theta,\phi}$, steps I, ACO iters T_{ACO}, ants m, budget B
Params: $\tau_0, \tau_{\min}, \tau_{\max}, \sigma$
$c \leftarrow$ random k-coloring; $c^\star \leftarrow c$
while elapsed $< B$ **do**
  for $t = 0, \ldots, I-1$ **do**
    $(v, q) \sim \pi_{\theta,\phi}(\cdot \mid s(c))$; $c \leftarrow c[v \mapsto q]$
    if $f(c) < f(c^\star)$ **then**
      $c^\star \leftarrow c$
    end if
  end for
  $P(v, q) \leftarrow \pi^{\mathrm{c}}_{\phi}(q \mid s(c), v)$
  $\tau \leftarrow \textsc{SoftInject}(P, \tau_0, \tau_{\min}, \tau_{\max}, \sigma)$
  $c \leftarrow \mathrm{ACO}(G, k, c, \tau, m, T_{\mathrm{ACO}})$
  if $f(c) < f(c^\star)$ **then**
    $c^\star \leftarrow c$
  end if
  if $f(c^\star) = 0$ **then**
    return $c^\star$
  end if
end while
return $c^\star$

policy provides more useful guidance than a random-policy prior under the same ACO pipeline.

To understand whether the total improvement in each RLACO episode comes mainly from the RL phase, mainly from the ACO phase, or from both, we record the conflict count at three checkpoints: before the RL phase (f_e^{start}), after the RL phase (f_e^{RL}), and after the ACO phase (f_e^{ACO}). The per-episode contribution of each phase is:

$$\Delta_e^{\mathrm{RL}} = \left[f_e^{\mathrm{start}} - f_e^{\mathrm{RL}}\right]_+, \qquad \Delta_e^{\mathrm{ACO}} = \left[f_e^{\mathrm{RL}} - f_e^{\mathrm{ACO}}\right]_+, \tag{3}$$

where $[x]_+ = \max(x, 0)$ ensures that a phase receives zero credit if it increases conflicts. Aggregating across all episodes yields cumulative contributions S_{RL} and S_{ACO}, from which the relative RL share is:

$$\mathrm{share}^{\mathrm{RL}} = \frac{S_{\mathrm{RL}}}{S_{\mathrm{RL}} + S_{\mathrm{ACO}}}, \qquad \mathrm{share}^{\mathrm{ACO}} = 1 - \mathrm{share}^{\mathrm{RL}}. \tag{4}$$

This decomposition helps characterize how the transferred policy participates in the search: whether it directly drives the improvement, or whether its main role is to provide a favorable starting point that the downstream ACO phase then exploits.

Table 1. DIMACS benchmark instances used in the experiments.

Instance	$\chi/k^\star$	Size	n	Transfer k	Family
DSJC250.5	?/28	Small	250	28	Random, .5
DSJC500.1	?/12	Medium	500	12	Random, .1
DSJC500.5	?/47	Medium	500	48	Random, .5
DSJC500.9	?/126	Medium	500	126	Random, .9
flat300_26_0	26/26	Small	300	26	Flat
flat300_28_0	28/28	Small	300	28	Flat
flat1000_76_0	76/81	Large	1000	–	Flat
le450_15c	15/15	Medium	450	15	Leighton
le450_15d	15/15	Medium	450	15	Leighton
le450_25c	25/25	Medium	450	25	Leighton
le450_25d	25/25	Medium	450	25	Leighton

4 Experimental Setup

4.1 Benchmark instances

The experiments use DIMACS graph-coloring instances from the DSJC, flat, and Leighton families. DSJC graphs are random graphs with the indicated density; the flat and Leighton families provide structured cases with different coloring behavior. Table 1 lists the instances and reference color numbers.

4.2 Protocols and metrics

The policy is trained only on Erdős–Rényi graphs and is frozen before evaluation. RLTabucol, pure Tabucol, and RLTabucol$_0$ use ten independent runs and a five-hour wall-clock budget. The primary metric is k_{best}, the smallest tested color number at which at least one run reaches zero conflicts; Success is the number of runs with score = 0.

RLACO and its controls use the fixed target k listed in Table 1, with the same ACO settings. RLACO$_0$ uses the random-policy control, pure ACO removes the RL phase, and RLACO$_{\text{config}}$ uses conflict-based constructed injection. We report the observed best and mean±standard deviation of the final running-best conflict. Phase shares count positive conflict reductions recorded during the RL and ACO phases; they are descriptive and are not causal attribution.

For RLACO+Tabucol, the RLACO stage is allowed the instance-dependent convergence time used in the transfer protocol. Its converged state is then passed to Tabucol without retraining, and Tabucol receives the same five-hour budget as pure Tabucol, RLTabucol, and RLTabucol$_0$. Terminal success is defined by score = 0. The terminal table follows the supplied results document. The selected transfer color is 48 for DSJC500.5, one above the reference value 47. Three selected terminal groups have nine raw summaries; these missing observations are recorded separately and are not counted as failures.

Table 2. RLTabucol compared with the random-policy baseline RLTabucol$_0$. k_{best} is the minimum tested k with at least one zero-conflict run; Success counts score = 0 runs.

Instance	$\chi/k^\star$	Size	k_{best} RLTabucol	Success RLTabucol	k_{best} RLTabucol$_0$	Success RLTabucol$_0$
`DSJC250.5`	?/28	Small	**28**	10/10	28	10/10
`DSJC500.1`	?/12	Medium	**12**	8/10	13	10/10
`DSJC500.5`	?/47	Medium	**49**	6/10	50	10/10
`DSJC500.9`	?/126	Medium	**128**	4/10	130	7/10
`le450_15c`†	15/15	Medium	16	10/10	15	10/10
`le450_15d`†	15/15	Medium	16	10/10	15	9/10
`le450_25c`	25/25	Medium	**26**	10/10	26	10/10
`le450_25d`	25/25	Medium	**26**	10/10	26	10/10
`flat300_26_0`	26/26	Small	**26**	10/10	26	10/10
`flat300_28_0`†	28/28	Small	31	10/10	30	1/10
`flat1000_76_0`	76/81	Large	**87**	1/10	90	10/10

5 Results

5.1 RLTabucol, RLTabucol$_0$, and pure Tabucol

The source-framework comparison establishes the role and the limit of the learned policy. RLTabucol achieves an equal or lower k_{best} than RLTabucol$_0$ on eight of eleven instances: four improvements and four ties, with three degradations. Pure Tabucol provides a second reference for the local-search component. The result supports a bounded claim: the policy supplies useful vertex–color guidance, but the direct actor–Tabucol handoff is not uniformly better than either baseline.

On three DSJC instances (DSJC500.1, DSJC500.5, and DSJC500.9) and on `flat1000_76_0`, RLTabucol reaches lower color budgets than RLTabucol$_0$ in the reported table. The three degradations occur on `le450_15c`, `le450_15d`, and `flat300_28_0`. These failures motivate the next diagnostic rather than being treated as noise: they show that immediate conflict reduction does not by itself guarantee a starting state that Tabucol can refine effectively.

5.2 Actor-only diagnosis

We use the original actor-only figure for `le450_15d` at $k = 15$. The actor reduces conflicts substantially from the initial infeasible coloring, but the trajectory plateaus near the value reported in the original analysis (approximately 232; the archived eight-run raw subset has a mean of 226.1, so this number is not presented as a new ten-run estimate). In contrast, the full RLTabucol runs reach the 5–21 conflict range because Tabucol performs fine-grained repair.

The diagnostic indicates a division of labor rather than a failed policy. The actor can produce a lower-conflict regime, while the much lower residual conflicts

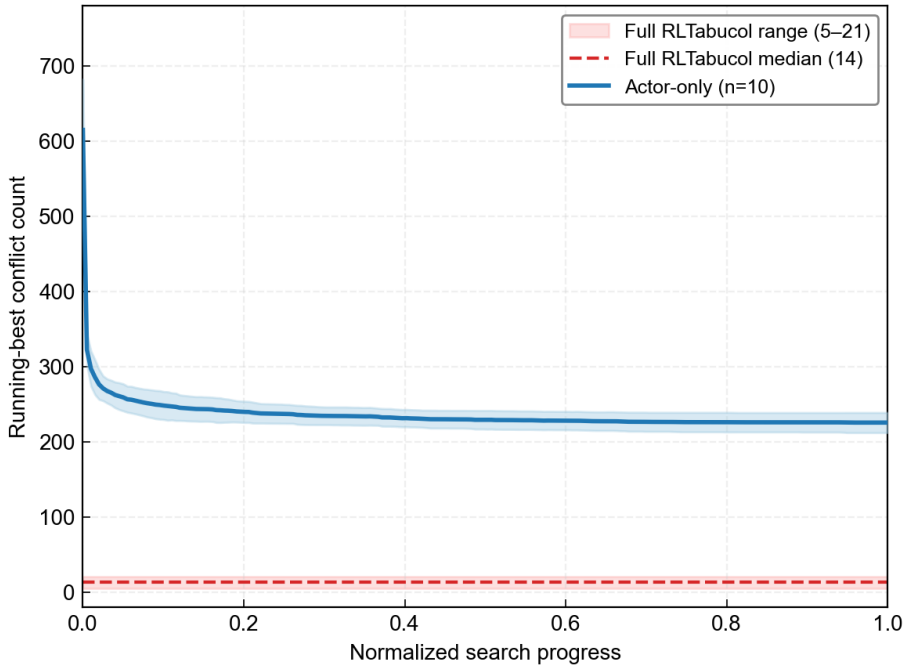


Fig. 2. Actor-only diagnostic on `le450_15d` ($k = 15$), retained from the original validated figure.

of the full pipeline indicate that Tabucol contributes substantially to fine repair. However, the observed pattern is consistent with the hypothesis that immediate conflict reduction dominates the useful signal, while states that are especially suitable for a later Tabucol search are less frequently produced. A direct actor–Tabucol handoff can therefore repeatedly return to a narrow near-feasible basin. This motivates adding population diversity while retaining the policy's preferences.

5.3 RLACO transfer and injection comparison

RLACO obtains lower observed best and mean conflicts than RLACO_0 on all ten transfer instances in the existing transfer table. Pure ACO is substantially weaker on the same fixed-k protocol, providing a framework-level reference for the benefit of the transferred policy. The three instances on which RLTabucol underperforms RLTabucol_0 also improve under RLACO, showing that policy value can survive a change in search framework.

The separate injection comparison evaluates RLACO against $\text{RLACO}_{\text{config}}$, whose prior is produced by conflict-based constructed injection. Soft injection is better on the observed best value on six instances, tied on one, and worse on

Table 3. RLACO transfer and injection controls. Entries are observed best conflict followed by mean ± standard deviation; lower is better.

Instance	RLACO	Config	$RLACO_0$	Pure ACO
DSJC250.5	16 (22.8+/-3.2)	21 (24.7+/-2.9)	70 (76.6+/-3.1)	69 (74.6+/-3.5)
DSJC500.1	48 (65.7+/-17.0)	57 (65.7+/-7.2)	136 (141.4+/-2.4)	123 (134.2+/-6.4)
DSJC500.5	74 (79.0+/-4.2)	76 (84.7+/-7.2)	225 (230.0+/-3.7)	224 (229.9+/-3.3)
DSJC500.9	42 (48.2+/-3.5)	42 (48.4+/-4.1)	196 (202.1+/-3.0)	192 (194.9+/-2.2)
flat300_26_0	93 (96.9+/-3.2)	86 (92.9+/-5.2)	187 (194.7+/-3.8)	179 (187.8+/-5.2)
flat300_28_0	58 (62.9+/-3.0)	57 (62.3+/-4.3)	151 (155.6+/-2.1)	136 (145.5+/-5.0)
le450_15c	222 (244.0+/-13.1)	221 (237.0+/-10.7)	295 (298.0+/-1.7)	293 (297.6+/-2.5)
le450_15d	229 (250.7+/-15.8)	234 (254.5+/-13.6)	290 (301.4+/-5.6)	295 (300.8+/-2.7)
le450_25c	43 (57.9+/-7.6)	60 (62.9+/-2.6)	63 (68.0+/-2.7)	68 (70.3+/-1.7)
le450_25d	44 (50.5+/-6.7)	61 (64.7+/-2.5)	66 (70.0+/-1.7)	70 (72.2+/-1.5)

three in the supplied comparison values. This result supports the use of the full color distribution as a graded prior, while also showing that the interface choice is instance-dependent.

5.4 Complementary RL and ACO roles

The phase-share analysis reports the fraction of positive conflict reduction recorded during each phase. We use these shares descriptively; they are not causal attribution and no inferential p-values are reported for them. On random and flat instances, most recorded reduction occurs during the RL phase. On the Leighton $k = 25$ instances, the ACO phase contributes most of the reduction, while the RL phase remains useful as a policy-informed starting prior. Thus the policy is not simply a direct conflict reducer: it can also give ACO a starting distribution that the population search can exploit. The two phases are complementary.

5.5 RLACO+Tabucol terminal optimization

The terminal experiment is the main contribution. The supplied terminal-results table compares pure Tabucol, RLTabucol, $RLTabucol_0$, and RLACO+Tabucol. RLACO is first run to its instance-dependent convergence point; the resulting state is then passed to Tabucol, which receives the same five-hour budget as the three Tabucol baselines. Success is defined by score = 0.

RLACO+Tabucol improves the best color number over pure Tabucol on DSJC500.9 (126 versus 128), le450_15c (15 versus 16), le450_15d (15 versus 16), and le450_25d (25 versus 26). It matches the best reported color number on DSJC250.5, DSJC500.1, DSJC500.5, le450_25c, flat300_26_0, flat300_28_0, and flat1000_76_0, while improving over pure Tabucol on the four instances listed above. The success counts reveal the qualification: several terminal cells have only 1–2 successful runs even when the color number is lower. The result is therefore an optimization improvement in the best attainable color budget, not a claim of uniformly higher success probability.

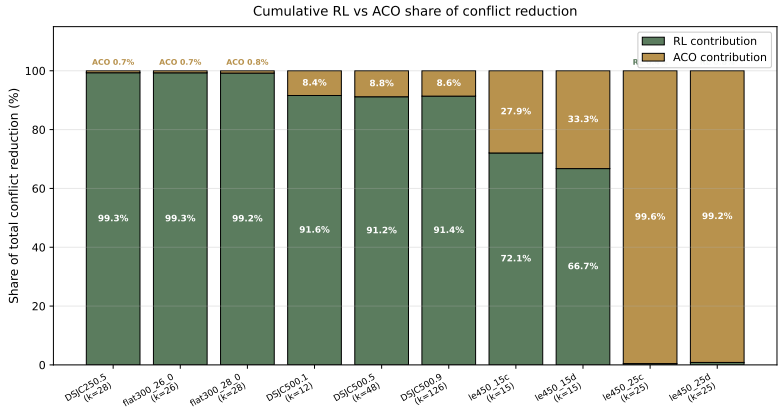


Fig. 3. Cumulative positive-reduction shares recorded during the RL and ACO phases. The shares describe observed reductions and do not imply causal attribution.

Table 4. Four-configuration terminal comparison from the supplied results document. Success counts use score = 0; time is the shortest observed wall-clock time reported for the corresponding configuration; this is a secondary observed time, whereas the protocol budgets are defined separately in Section 4.

Instance	Pure Tabucol			RLTabucol			RLTabucol_0			RLACO+Tabucol		
	k_{best}	Success	Time(s)	k_{best}	Success	Time(s)	k_{best}	Success	Time(s)	k_{best}	Success	Time(s)
DSJC250.5	28	10/10	46	28	10/10	243	28	10/10	2	**28**	9/10	**1.546**
DSJC500.1	12	7/10	6001	12	8/10	**936**	13	10/10	0.3	**12**	9/10	1357.971
DSJC500.5	49	10/10	1520	49	6/10	2375	50	10/10	101	**49**	10/10	**424.005**
DSJC500.9	128	10/10	8522	128	4/10	3062	130	7/10	1080	**126**	1/10	**7330.602**
le450_15c	16	10/10	45	16	10/10	4	15	10/10	273	**15**	2/10	**130.846**
le450_15d	16	9/10	303	16	10/10	32	15	9/10	932	**15**	1/10	**578.453**
le450_25c	26	9/10	12	26	10/10	44	26	10/10	3	**26**	10/10	**1.456**
le450_25d	26	10/10	15	26	10/10	44	26	10/10	1	**25**	2/10	**8480.672**
flat300_26_0	26	10/10	216	26	10/10	229	26	10/10	134	**26**	10/10	**13.312**
flat300_28_0	30	1/10	9454	31	10/10	150	30	1/10	14021	**30**	1/10	**4788.221**
flat1000_76_0	88	1/10	16485	**87**	1/10	**8920**	90	10/10	303	**87**	1/10	13324.46

The raw terminal archive is incomplete for three selected groups: DSJC250.5 at $k = 28$, DSJC500.5 at $k = 49$, and DSJC500.9 at $k = 126$ each have nine summaries rather than ten. The numerical table follows the supplied document as requested, while these missing seeds are recorded as a limitation and are not counted as failures.

6 Discussion and Limitations

The experiments support a staged interpretation. The source-framework comparison shows that the policy contains useful local information, but a direct handoff to Tabucol is not consistently effective. The actor-only diagnostic explains why:

the actor reduces conflicts at a coarse scale, whereas Tabucol supplies the dominant fine repair and can stagnate when the starting states lack suitable diversity.

The transfer experiments provide that missing diversity. Soft pheromone injection preserves a distribution over colors, so ACO can explore multiple constructions while remaining guided by the learned preferences. The phase-share results show that the policy's role changes across instances. It directly accounts for much of the observed reduction on some graphs; on others, ACO accounts for more of the reduction after receiving a policy-informed prior. These shares are descriptive rather than causal.

RLACO+Tabucol is the main optimization result. The terminal table shows lower best color budgets than pure Tabucol on four hard instances and the best or tied-best budget on the remaining listed cases. The improvement is concentrated in the best attainable color number; success counts are not uniformly higher, which is why the result should be interpreted as a stronger initialization and optimization mechanism rather than universal robustness.

The study has four limitations. Three selected terminal groups lack one raw seed, and the supplied document is therefore the authority for the reported table while the archive is completed. Only one frozen policy and one soft-injection strength are tested. The benchmark set is moderate and the transfer target is ACO. Finally, the RLACO stage uses instance-dependent convergence time. Future work should complete the missing seeds, test fixed total budgets, evaluate multiple policies and injection strengths, and extend the transfer target.

7 Conclusion

We studied a learned vertex–color policy across two search frameworks and a terminal hybrid. RLTabucol is competitive with its random-policy baseline on most instances, but the actor-only diagnostic shows why direct handoff to Tabucol can fail: coarse conflict reduction does not provide enough diversity for the final local repair.

Soft transfer to ACO addresses this mismatch. RLACO improves the reported random-policy and pure-ACO references, and the injection comparison supports retaining the policy's full color distribution rather than using only a conflict-based constructed coloring. The phase-share analysis shows that RL and ACO play complementary roles.

The main contribution is RLACO+Tabucol. After instance-dependent RLACO convergence, the transferred state is refined by Tabucol under the same five-hour budget used by the source baselines. The terminal results improve the best color budget on DSJC500.9, le450_15c, le450_15d, and le450_25d, and reach the best or tied-best budget on the other listed instances, although several cells have low success counts. The evidence supports using policy transfer to create diverse, policy-informed starting states for a strong downstream local search.

Future work should complete the missing terminal seeds, evaluate fixed-budget protocols, and test the mechanism with additional policies, injection strengths, and target metaheuristics.

References

1. Cai, T., Kraska, T., Alizadeh, M., Karbasi, A.: Reinforcement learning driven heuristic optimization. arXiv preprint arXiv:1906.06639 (2019)
2. Costa, D., Hertz, A.: Ants can colour graphs. Journal of the Operational Research Society **48**(3), 295–305 (1997). https://doi.org/10.1057/palgrave.jors.2600357
3. Frijters, M.: Reinforcement Learning for Improved Local Search: Applied to the Graph Coloring Problem. Master's thesis, KTH Royal Institute of Technology, Stockholm, Sweden (2023), urn:nbn:se:kth:diva-331002
4. Galinier, P., Hao, J.K.: Hybrid evolutionary algorithms for graph coloring. Journal of Combinatorial Optimization **3**(4), 379–397 (1999). https://doi.org/10.1023/A:1009823419804
5. Galinier, P., Hertz, A., Zufferey, N.: An adaptive memory algorithm for the k-coloring problem. Discrete Applied Mathematics **156**(2), 267–279 (2008). https://doi.org/10.1016/j.dam.2006.03.067
6. Hertz, A., de Werra, D.: Using tabu search techniques for graph coloring. Computing **39**(4), 345–351 (1987). https://doi.org/10.1007/BF02239976
7. Huang, J., Patwary, M., Diamos, G.: Coloring big graphs with AlphaGoZero. arXiv preprint arXiv:1902.10162 (2019)
8. Langedal, K., Manne, F.: Graph neural networks as ordering heuristics for parallel graph coloring. arXiv preprint arXiv:2408.05054 (2024), presented at the ALENEX 2024 workshop
9. Malaguti, E., Toth, P.: A survey on vertex coloring problems. International Transactions in Operational Research **17**(1), 1–34 (2010). https://doi.org/10.1111/j.1475-3995.2009.00696.x
10. Schulman, J., Wolski, F., Dhariwal, P., Radford, A., Klimov, O.: Proximal policy optimization algorithms. arXiv preprint arXiv:1707.06347 (2017)
11. Stützle, T., Hoos, H.H.: MAX-MIN ant system. Future Generation Computer Systems **16**(8), 889–914 (2000). https://doi.org/10.1016/S0167-739X(00)00043-1
12. Sun, Y., Ernst, A.T., Li, X., Weiner, J.: Boosting ant colony optimization via solution prediction and machine learning. Computers & Operations Research **143**, 105769 (2022). https://doi.org/10.1016/j.cor.2022.105769
13. Wang, X., Yan, X., Jin, Y.: A graph neural network with negative message passing and uniformity maximization for graph coloring. Complex & Intelligent Systems **10** (2024). https://doi.org/10.1007/s40747-024-01355-w
14. Watkins, G., Montana, G., Branke, J.: Generating a graph colouring heuristic with deep Q-learning and graph neural networks. In: Learning and Intelligent Optimization (LION). Springer (2023). https://doi.org/10.1007/978-3-031-44505-7_33
15. Yang, W., Li, Z.: Graph coloring problem solving using monte carlo tree search and deep reinforcement learning. Journal of Combinatorial Optimization **50**(2) (2025). https://doi.org/10.1007/s10878-025-01338-8
16. Ye, H., Wang, J., Cao, Z., Liang, H., Li, Y.: DeepACO: Neural-enhanced ant systems for combinatorial optimization. In: Advances in Neural Information Processing Systems (NeurIPS) (2023)

A Experimental details

A.1 Algorithm pseudocode

Algorithms 3 and 4 give the pseudocode for the Tabucol and ACO procedures referenced in Sections 3.2 and 3.6.

Algorithm 3 Tabucol procedure

Input: $G = (V, E)$, colors k, coloring c_0, limit T
$c \leftarrow c_0$; $c^\star \leftarrow c_0$; init tabu list
for $t = 1, \dots, T$ **do**
  $(v, q) \leftarrow$ best admissible move (v conflicting, $q \neq c(v)$)
  Add $(v, c(v))$ to tabu list; $c \leftarrow c[v \mapsto q]$
  if $f(c) < f(c^\star)$ **then**
    $c^\star \leftarrow c$
  end if
  if $f(c^\star) = 0$ **then**
    return $c^\star$
  end if
end for
return $c^\star$ w.p. $1 - \beta$; else c

Algorithm 4 ACO procedure

Input: $G = (V, E)$, k, ants m, iters T, pheromone τ
$c^\star \leftarrow$ random k-coloring
for $t = 1, \dots, T$ **do**
  for each ant $a = 1, \dots, m$ **do**
    Construct c_a via pseudo-random-proportional rule
    if $f(c_a) < f(c^\star)$ **then**
      $c^\star \leftarrow c_a$
    end if
  end for
  Evaporate; reinforce elite; clip τ to $[\tau_{\min}, \tau_{\max}]$
  if $f(c^\star) = 0$ **then**
    return $c^\star$
  end if
end for
return $c^\star$

A.2 Parameter settings

Tables 5–7 report the RLTabucol, RLACO, and PPO settings used in the experiments.

Table 5. RLTabucol parameters.

Parameter	Training	Testing
RL steps I	300	300
Init coloring	Uniform rand.	Uniform rand.
Max epoch	50	No limit
Tabucol iters T	5000	10000
Tabu tenure A, α	10, 1.2	10, 1.2
Return prob. β	0.2	0.2
Run count	PPO sched.	10
Time budget	No limit	18000s

Table 6. RLACO parameters.

Parameter	Value
Target k	Fixed (Tbl. 1)
Ants m	40
ACO iters T_{ACO}	50
$\alpha_{\text{ACO}}, \beta_{\text{ACO}}$	1.0, 3.0
Evaporation ρ	0.20
Q, q_0	1.0, 0.85
Elite, weight	3, 2.0
$\tau_0, \tau_{\min}, \tau_{\max}$	1.0, 0.05, 8.0
Injection σ	8.0
ACO local search	Disabled

Table 7. PPO training parameters.

Parameter	Value
Optimizer, LR	Adam, 3×10^{-4}
Clip ϵ	0.2
Buffer, batch	30000, 512
Envs, test eps	10, 10
Steps/epoch	50000
Steps/collect	2000
Repeat/collect	10
γ, entropy	Tianshou defaults
Value-loss coef.	Tianshou defaults